

Native Spark ve GE-on-Spark Validation Karşılaştırması

LOKAL
PERFORMANS
RAPORU

10 GiB sentetik Parquet verisinin ortak Spark processing adımı ve aynı processed veri üzerinde native Spark ile Great Expectations validation maliyetleri incelenmiştir.

83.9 milyon	11.902 GiB	3.62x
Processed satır sayısı	Validation girdisi	Sıcak koşuda GE / native süre oranı

Kapsam

GE bir processing motoru değildir. Bu nedenle processing tek bir ortak Spark adımı olarak uygulanmış; performans karşılaştırması yalnızca validation katmanında yapılmıştır. Rapor yalnızca gerçekten çalıştırılan başarılı veri koşullarını içerir.

Benchmark koşulları: 21 Ağustos 2026 | Rapor: 24 Ağustos 2026

1. Amaç ve karşılaştırma sınırı

Bu çalışma, büyük bir processed Parquet datasetinde aynı kalite kurallarının native Spark ve Great Expectations Spark backend'i ile çalıştırılma maliyetini karşılaştırır. GE veri dönüştürme aracı olmadığı için processing iki alternatif arasında karşılaştırılmamış, her iki validator için ortak hazırlanmıştır.

Karşılaştırılan bölüm Ortak processing çıktısı -> aynı Spark DataFrame -> aynı 16 mantıksal validation kuralı. Ölçülen fark validation framework ve Spark execution planı maliyetidir.

2. Çalışma ortamı

Özellik	Değer
İşletim sistemi	Windows 11 - 10.0.26200
Python	3.12.10
Java	21.0.12.1 LTS
Spark	4.1.1
Great Expectations	1.20.0
Spark master	local[8]
Driver memory	4 GiB
Makine	28 logical core
Veri konumu	Yerel disk

3. Test verisi

Özellik	Girdi V1
Boyut	10.002 GiB - 10,739,950,326 byte
Satır	83,886,080
Sütun	16 adet double feature
Row group	80
Satır limiti	Yok - tam veri işlendi

Windows'ta PySpark'ın yerel Hadoop filesystem katmanının permission çağrıları için küçük bir Java adaptörü kullanılmıştır. Adaptör yalnızca yerel filesystem uyumluluğunu sağlar; transformation, validation veya Parquet içeriğini değiştirmez.

4. Ortak Spark processing akışı

Spark oturumu local[8] ile açılmış; adaptive query execution etkinleştirilmiş, maksimum dosya partition boyutu 128 MiB, shuffle partition sayısı 56 ve Parquet çıktı sıkıştırması Snappy olarak ayarlanmıştır.

1. Beklenen 16 feature kolonunun varlığı kontrol edildi.
2. Feature kolonlarında null bulunan satırlar kaldırıldı.
3. On altı feature'ın ortalamasıyla feature_mean üretildi.
4. En büyük ve en küçük feature farkıyla feature_spread üretildi.
5. İlk üç feature için 0.5 / 0.3 / 0.2 ağırlıklarıyla risk_score hesaplandı.
6. Risk skoru low, medium ve high değerlerinden oluşan risk_band kolonuna çevrildi.
7. Çıktı overwrite modunda birden fazla part Parquet dosyası olarak yazıldı.

Ölçüm	Sonuç
Girdi	10.002 GiB
Çıktı	11.902 GiB
Girdi / çıktı satırı	83,886,080 / 83,886,080
Çıktı row group	160
Spark başlangıcı	3.562 sn
Read + transform planı	1.397 sn
Process + write	627.490 sn
Toplam	640.973 sn - 10 dk 41 sn

Spark lazy evaluation

Read + transform planının 1.397 saniye sürmesi gerçek verinin bu sürede işlendiği anlamına gelmez. spark.read ve transformation çağrıları execution planını oluşturur. Gerçek okuma, dönüşüm ve yazma işi processed.write çağrısındaki 627.490 saniyede gerçekleşmiştir.

Çıktı bir Parquet dataset klasörüdür ve Spark tarafından üretilen part dosyalarından oluşur. Bu testte tarih/uçuş gibi bir kolonla partitionBy uygulanmamıştır.

5. Ortak validation kuralları

- Kolonların beklenen sırayla eşleşmesi.
- Satır sayısının 83,886,080 olması.
- feature_01, feature_08, feature_16, feature_mean, feature_spread ve risk_score kolonlarının null içermemesi.
- Aynı altı sayısal kolonun 0 ile 1 arasında olması.
- risk_band kolonunun null içermemesi.
- risk_band değerlerinin yalnızca low, medium ve high kümesinde bulunması.

Toplam 16 mantıksal kural iki implementasyonda da aynı processed DataFrame üzerinde uygulanmıştır.

6. İki validation yaklaşımı

Native Spark

Null sayıları, minimumlar, maksimumlar, satır sayısı ve kategori dışı değerler tek bir aggregation listesinde birleştirilmiştir. Sonuçlardan özel bir JSON kural listesi oluşturulmuştur.

GE-on-Spark

Ephemeral GE context, Spark datasource, whole-dataframe batch ve 16 expectation içeren bir suite oluşturulmuştur. Validation BASIC result formatıyla çalıştırılmıştır. GE datasource için persist=False ayarlanmış; benchmark sırasında zorunlu DataFrame persist/cache kullanılmamıştır.

Eşit veri girişi

İki validator aynı SparkSession içinde aynı logical DataFrame planını kullanmıştır. Scriptte DataFrame persist edilmemiştir; buna rağmen işletim sistemi ve dosya okuma cache'leri çalışma sırasını etkileyebilir.

7. Validation sonuçları

İlk validator'ın soğuk dosya I/O maliyetini taşıması ve ikinci validator'ın sıcak cache etkisinden yararlanması ihtimalini göstermek için iki ayrı çalışma sırası kullanılmıştır.

Koşu	Validator	Sıra	Süre	Job	Stage	Sonuç
Native-first	Native Spark	1	144.240 sn	2	3	16/16 geçti
Native-first	GE-on-Spark	2	12.505 sn	64	68	16/16 geçti
GE-first sıcak tekrar	GE-on-Spark	1	13.019 sn	64	68	16/16 geçti
GE-first sıcak tekrar	Native Spark	2	3.592 sn	2	3	16/16 geçti

Sıcak tekrar süreleri

Native Spark

3.59 sn

GE-on-Spark

13.02 sn

Sıcak tekrar - düşük değer daha iyidir

Ölçülen fark

Sıcak tekrar değerlerinde native Spark 3.592 saniye, GE-on-Spark 13.019 saniye sürmüştür. Bu koşuda GE yaklaşık 3.62 kat daha yavaş ve yaklaşık 9.43 saniye daha uzun çalışmıştır. Buna karşılık standart expectation sonucu ve ayrıntılı JSON raporu hazır üretmiştir.

Native-first koşusundaki 144.240 saniyelik native süre framework overhead'i olarak yorumlanmamalıdır. Bu değer ilk büyük veri taramasının soğuk I/O maliyetini taşımaktadır. İki sıra sonuçları cache etkisinin benchmarkı ne kadar değiştirebildiğini göstermektedir.

8. Trade-off ve nihai karar

Performans	Tek aggregation planı; sıcak koşuda 3,59 sn	Expectation değerlendirmeleri; sıcak koşuda 13,02 sn
Spark planı	2 job / 3 stage	64 job / 68 stage
Kural tanımı	Aggregation ve raporlama kodu elle yazılır	Declarative expectation suite
Rapor	Özel JSON şeması tarafımızdan üretilir	Standart validation result ve expectation detayları
Bakım	Daha az dependency, daha fazla özel kod	GE/Spark uyumluluğu; daha az raporlama kodu

9. Seçilen mimari

GE + Spark kullanılmasına karar verilmiştir.

Processing adımları doğrudan native Spark dönüşümleriyle; validation adımları Spark execution engine üzerinde Great Expectations ile çalışacaktır. GE'nin ek yürütme maliyeti, standart ve denetlenebilir kalite raporu ihtiyacı karşılığında kabul edilmiştir.

- Processing: native Spark dönüşümleri
- Validation: Great Expectations + Spark execution engine
- Kalite çıktısı: GE validation result; ileride Data Docs/checkpoint raporları

10. Geçerlilik ve sınırlamalar

- Test lokal Spark local[8] üzerinde yapılmıştır; dağıtık cluster davranışını ölçmez.
- Processed veri yerel diskten okunmuştur; MinIO ve network I/O karşılaştırmaya dahil değildir.
- Her execution order yalnızca bir kez çalıştırılmış; ortalama, medyan ve standart sapma hesaplanmamıştır.
- İşletim sistemi dosya cache'i temizlenmemiştir; validator sırası sonuçları belirgin biçimde etkilemiştir.
- Sentetik veri uniform dağılımlıdır ve çalıştırılan iki validation koşusunda bütün kurallar başarılıdır.
- Rapor yalnızca çalıştırılan pozitif validation koşularını kapsar; hata enjeksiyonu sonucu içermez.
- Kesin süre oranı yerine job/stage farkı ve raporlama trade-off'u karar girdisi olarak değerlendirilmiştir.

11. Kanıt dosyaları

Processing: scripts/benchmark_spark_processing.py

Validation: scripts/benchmark_spark_validation.py

Runtime sonuçları: reports/benchmarks/ altındaki processing ve validation JSON/CSV dosyaları